

Packager Kubernetes avec Helm

Charts, values, templates & releases — déployer et versionner ses applications K8s

Formation DevOps • avec illustrations

Formateur : Haithem Mihoubi

7 modules • fil rouge : un chart nginx • 2026

Table des matières

00	Introduction : pourquoi Helm ?	3
01	Charts, Releases & Repositories	6
02	La structure d'un chart	9
03	Templates & values : le cœur de Helm	13
04	Releases : installer, mettre à jour, rollback	17
05	Dépendances & repositories	20
06	Bonnes pratiques, débogage & checklist	23

Introduction : pourquoi Helm ?

1. Le problème : des YAML partout

Déployer une application sur Kubernetes, c'est écrire beaucoup de manifestes : Deployment, Service, ConfigMap, Secret, Ingress... Très vite, plusieurs douleurs apparaissent :

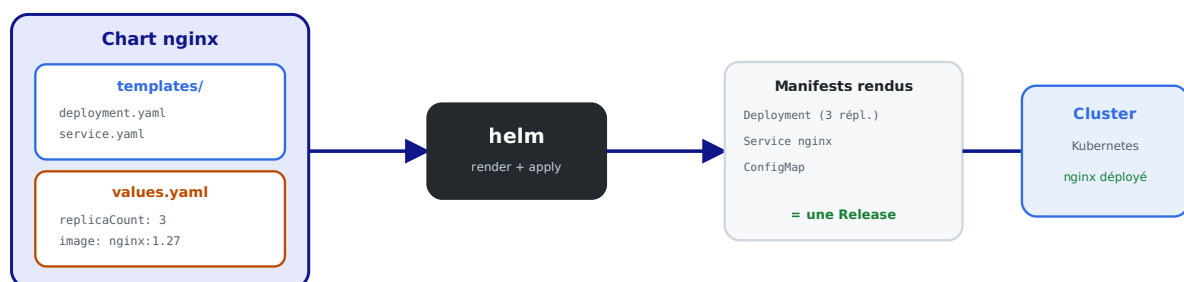
- **La duplication** : on copie-colle les mêmes fichiers pour `dev`, `staging`, `prod`, en changeant juste deux ou trois valeurs (nombre de réplicas, image, domaine).
- **Pas de paramétrage** : changer le tag de l'image impose d'éditer le YAML à la main.
- **Pas de versionnement applicatif** : impossible de dire « installe la v1.2 de mon appli » ou « reviens à la version précédente » d'un coup.
- **Pas de cycle de vie** : installer, mettre à jour, désinstaller un ensemble cohérent d'objets relève du bricolage avec `kubectl apply / delete`.

En clair : `kubectl` sait appliquer des fichiers, mais ne sait pas **packager**, **paramétrer** ni **versionner** une application. Il manque un gestionnaire de paquets.

2. La solution : Helm

Helm est le **gestionnaire de paquets de Kubernetes** (l'équivalent d'`apt` ou `npm`, mais pour des applications K8s). Il empaquette tous les manifestes dans un **chart** paramétrable, l'installe en tant que **release** versionnée, et gère son cycle de vie.

Helm : le gestionnaire de paquets de Kubernetes



Un Chart (modèles + valeurs) → helm les assemble → une Release versionnée tourne dans le cluster.

Un Chart (modèles + valeurs) → helm les assemble → une Release versionnée tourne dans le cluster.

3. Les trois mots du vocabulaire Helm

Terme	Définition	Analogie (npm)
Chart	Un paquet : modèles de manifestes + valeurs par défaut	le <code>package</code>
Release	Une instance installée d'un chart dans le cluster	l'appli installée
Repository	Un dépôt d'où l'on télécharge des charts	le registre npm

Un même chart peut donner **plusieurs releases** (ex. : `nginx-dev` et `nginx-prod`, chacune avec ses propres valeurs).

4. Ce que Helm apporte concrètement

Sans Helm (<code>kubectl</code>)	Avec Helm
Copier-coller des YAML par environnement	Un chart + des fichiers de valeurs
Éditer le YAML pour changer une valeur	<code>--set image.tag=1.28</code>
Suivi des versions « à la main »	<code>helm history</code> , révisions automatiques
Rollback manuel et risqué	<code>helm rollback</code> en une commande
Installer 50 objets un par un	<code>helm install</code> déploie tout d'un coup
Réutiliser le travail des autres	<code>helm install bitnami/nginx</code> depuis un repo

5. Notre fil rouge : un chart nginx

Comme pour le cours Kubernetes, on illustre **chaque concept** avec **nginx**. On va :

1. comprendre l'**architecture** de Helm (chart, release, repo) ;
2. créer un chart nginx et explorer sa **structure** ;
3. **templatiser** les manifestes avec des valeurs (`values.yaml`) ;
4. installer, **mettre à jour** et faire un **rollback** d'une release ;
5. gérer les **dépendances** et les **repositories** ;
6. appliquer les **bonnes pratiques** (lint, dry-run, packaging).

6. Installer Helm

```
# Linux (script officiel)
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Vérifier
helm version          # v3.x

# Helm utilise le contexte kubectl courant
kubectl config current-context
```

Helm 3 (la version actuelle) n'a **plus** de composant serveur (« Tiller ») : c'est un simple binaire côté client qui parle à l'API Kubernetes. Plus simple et plus sûr que Helm 2.

Premier essai pour se faire une idée :

```
helm create demo          # génère un chart d'exemple complet
ls demo/                  # Chart.yaml values.yaml templates/ charts/
```

Dans le module suivant, on décortique ces concepts un par un.

Charts, Releases & Repositories

Avant de manipuler Helm, posons précisément les trois concepts qui structurent tout : le **Chart**, la **Release** et le **Repository**.

1. Le Chart : le paquet

Un **chart** est un dossier (ou une archive `.tgz`) qui contient **tout le nécessaire** pour déployer une application :

- des **modèles** de manifestes Kubernetes (`templates/`) ;
- des **valeurs par défaut** configurables (`values.yaml`) ;
- des **métadonnées** (`Chart.yaml` : nom, version, description) ;
- éventuellement des **dépendances** (sous-charts).

Un chart est **inerte** : c'est une recette. Tant qu'on ne l'installe pas, rien ne tourne. On peut le partager, le versionner, le publier sur un repository.

2. La Release : l'instance installée

Quand on **installe** un chart, Helm crée une **release** : une instance nommée et **versionnée** de ce chart dans le cluster.

```
helm install nginx-prod ./nginx          # release "nginx-prod" à partir du chart ./nginx
helm install nginx-dev ./nginx --set replicaCount=1 # autre release, autres valeurs
```

Points clés :

- Un **même chart** → **plusieurs releases** (dev, prod, par client...), chacune isolée.
- Chaque release a un **historique de révisions** : install = révision 1, chaque upgrade incrémente. C'est ce qui permet le **rollback**.
- Helm stocke l'état de chaque release dans le cluster (dans des Secrets du namespace).

```
helm list          # lister les releases installées
helm status nginx-prod # état détaillé d'une release
helm history nginx-prod # toutes les révisions
```

3. Le Repository : le dépôt de charts

Un **repository** est un serveur HTTP qui héberge des charts packagés, avec un `index.yaml` qui les catalogue. C'est l'équivalent d'un registre de paquets.

```
# Ajouter un repository public
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Chercher et installer un chart tout prêt
helm search repo nginx
helm install mon-nginx bitnami/nginx
```

On n'écrit pas toujours ses charts : pour des briques standard (nginx, PostgreSQL, Redis, Prometheus...), on **réutilise** des charts publics éprouvés, en surchargeant juste les valeurs qui nous intéressent.

4. Le schéma mental complet

```
Repository —(helm pull / install)—> Chart —(helm install + values)—> Release —> Objets
(catalogue)                        (la recette)                        (l'instance)      (Pods, Sé
```

Question	Réponse
« Où je trouve des charts ? »	dans un Repository
« Qu'est-ce que je télécharge ? »	un Chart
« Qu'est-ce qui tourne dans le cluster ? »	une Release (instance du chart)
« Comment je change la config ? »	les values passées à l'install

5. Helm vs kubectl vs Kustomize

Outil	Rôle	Quand
kubectl	applique des manifestes bruts	objets simples, apprentissage
Kustomize	superpose des patches sur des YAML de base	variations légères, intégré à kubectl
Helm	package + templatisé + versionne + cycle de vie	applications réutilisables, multi-environnements, écosystème de charts

Helm et Kustomize ne s'excluent pas. Helm brille pour packager/distribuer et pour le cycle de vie (upgrade/rollback) ; Kustomize pour de simples surcharges. Beaucoup d'équipes utilisent Helm pour les briques tierces et Kustomize pour leurs ajustements.

6. Les commandes à connaître dès maintenant

```
helm create <nom>           # créer un chart squelette
helm lint <chart>           # vérifier la validité du chart
helm template <chart>       # rendre les manifestes SANS installer (debug)
helm install <release> <chart> # installer
helm upgrade <release> <chart> # mettre à jour
helm rollback <release> <rev>  # revenir à une révision
helm uninstall <release>       # désinstaller (supprime tous les objets)
helm list / helm history <release> # observer
```

Dans le module suivant, on ouvre un chart et on examine **chaque fichier**.

La structure d'un chart

Un chart est avant tout une **arborescence de fichiers** bien définie. Helm sait exactement où chercher chaque chose.

L'arborescence d'un chart Helm

► nginx/	le dossier du chart
Chart.yaml	métadonnées : nom, version
values.yaml	valeurs par défaut (configurables)
► templates/	les modèles de manifestes
deployment.yaml	Deployment nginx (templatisé)
service.yaml	Service nginx (templatisé)
_helpers.tpl	fonctions/labels réutilisables
NOTES.txt	message affiché après install
► charts/	sous-charts (dépendances)

Chart.yaml (métadonnées), values.yaml (valeurs), templates/ (modèles), charts/ (dépendances).

1. L'arborescence générée par `helm create`

```
helm create nginx
```

```
nginx/
├── Chart.yaml          # métadonnées du chart
├── values.yaml         # valeurs par défaut (configurables)
├── templates/         # les modèles de manifestes
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── _helpers.tpl    # fonctions/labels réutilisables
│   ├── NOTES.txt      # message affiché après l'install
│   └── ...
└── charts/            # sous-charts (dépendances)
```

2. `Chart.yaml` — la carte d'identité

```
apiVersion: v2          # v2 = Helm 3
name: nginx              # nom du chart
description: Un chart pour déployer nginx
type: application        # "application" ou "library"
version: 1.0.0           # version DU CHART (SemVer)
appVersion: "1.27"       # version de l'APPLICATION packagée (nginx)
```

Deux versions à ne pas confondre : - `version` : la version **du chart** (de l'emballage). On l'incrémente à chaque modification du chart. - `appVersion` : la version de **l'application** embarquée (ici nginx 1.27). Purement informative.

3. `values.yaml` — les valeurs par défaut

C'est **le** fichier de configuration. Il définit toutes les valeurs paramétrables et leurs **défauts**. L'utilisateur les surcharge à l'installation.

```
replicaCount: 3

image:
  repository: nginx
  tag: "1.27"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

resources:
  requests:
    cpu: 100m
    memory: 64Mi
```

Toute valeur ici est accessible dans les templates via `{{ .Values.xxx }}`. Par exemple `{{ .Values.replicaCount }}` ou `{{ .Values.image.tag }}`.

4. `templates/` — les modèles

Ce dossier contient les manifestes Kubernetes, mais **templatisés** : ils contiennent des expressions `{{ ... }}` qui seront remplacées par les valeurs au moment de l'install.

`templates/deployment.yaml` (extrait) :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.port }}
```

5. `_helpers.tpl` — les fonctions réutilisables

Les fichiers commençant par `_` ne génèrent **pas** de manifeste : ils définissent des **morceaux réutilisables** (nommage, labels communs). Exemple :

```
{{- define "nginx.labels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}
```

On les inclut ensuite dans les templates avec `{{ include "nginx.labels" . }}` — pour ne pas répéter les mêmes labels dans chaque fichier.

6. `NOTES.txt` — le message d'accueil

Affiché **après** `helm install`, c'est un mode d'emploi rapide pour l'utilisateur (souvent généré dynamiquement). Exemple : « Votre nginx est accessible via... ».

7. Les objets de contexte intégrés

Dans les templates, Helm fournit des objets prêts à l'emploi :

Objet	Contenu	Exemple
<code>.Values</code>	les valeurs de <code>values.yaml</code> (+ surcharges)	<code>.Values.replicaCount</code>
<code>.Release</code>	infos sur la release	<code>.Release.Name</code> , <code>.Release.Namespace</code>
<code>.Chart</code>	infos de <code>Chart.yaml</code>	<code>.Chart.Name</code> , <code>.Chart.Version</code>
<code>.Files</code>	accès aux fichiers du chart	<code>.Files.Get "config.txt"</code>
<code>.Capabilities</code>	capacités du cluster	versions d'API disponibles

8. Vérifier la structure

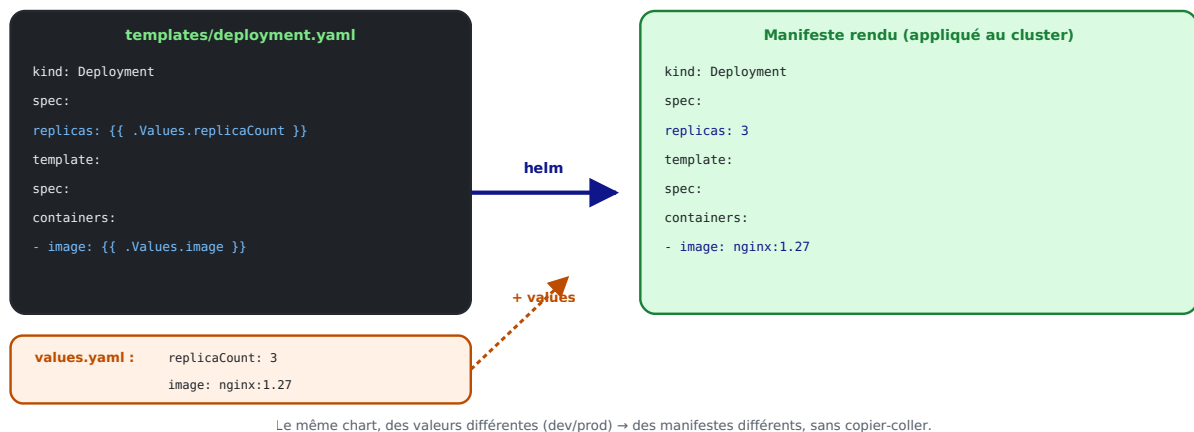
```
helm lint nginx          # erreurs/avertissements de structure
helm show chart nginx    # afficher Chart.yaml
helm show values nginx   # afficher les valeurs par défaut
```

À retenir : un chart, c'est `Chart.yaml` (qui), `values.yaml` (quoi configurer) et `templates/` (comment générer les manifestes). Le module suivant montre le cœur du sujet : le **templating**.

Templates & values : le cœur de Helm

C'est la puissance de Helm : **un seul** modèle, paramétré par des **valeurs**, génère des manifestes différents selon l'environnement — **sans copier-coller**.

Le templating : un modèle + des valeurs = un manifeste



Le même chart, des valeurs différentes (dev/prod) → des manifestes différents.

1. La syntaxe de base : {{ ... }}

Helm utilise le moteur de **templates Go**. Tout ce qui est entre `{{ }}` est **évalué** puis remplacé.

```
spec:
  replicas: {{ .Values.replicaCount }}
  # si values.yaml contient replicaCount: 3 → replicas: 3
```

2. Injecter des valeurs

```
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
# → image: "nginx:1.27"

metadata:
  name: {{ .Release.Name }}-nginx
  # → name: nginx-prod-nginx (si la release s'appelle "nginx-prod")
```

3. Surcharger les valeurs à l'installation

Trois façons de fournir des valeurs, par ordre de priorité croissante :

```
# 1) valeurs par défaut du chart (values.yaml)           – priorité basse
helm install nginx-prod ./nginx

# 2) un fichier de valeurs spécifique
helm install nginx-prod ./nginx -f values-prod.yaml

# 3) en ligne de commande (--set)                       – priorité haute
helm install nginx-prod ./nginx --set replicaCount=5 --set image.tag=1.28
```

Le multi-environnement, le vrai intérêt : on garde **un** chart et on a des fichiers `values-dev.yaml`, `values-staging.yaml`, `values-prod.yaml`. Seules les valeurs changent.

```
# values-prod.yaml
replicaCount: 5
image:
  tag: "1.28"
resources:
  limits:
    cpu: 500m
    memory: 256Mi
```

4. Les conditions

On inclut/exclut un bloc selon une valeur :

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}-nginx
# ...
{{- end }}
```

→ l'Ingress n'est généré **que si** `ingress.enabled: true`. Pratique pour activer une fonctionnalité selon l'environnement.

5. Les boucles

Pour générer une liste (variables d'env, ports, hosts...) :

```
env:
{{- range .Values.extraEnv }}
  - name: {{ .name }}
    value: {{ .value | quote }}
{{- end }}
```

```
# values.yaml
extraEnv:
  - name: ENV
    value: production
  - name: LOG_LEVEL
    value: warn
```

6. Les fonctions et les « pipes »

Le caractère `|` enchaîne des **fonctions** (comme un pipe shell) :

Expression	Résultat
<code>{{ .Values.name upper }}</code>	met en MAJUSCULES
<code>{{ .Values.name default "nginx" }}</code>	valeur par défaut si vide
<code>{{ .Values.tag quote }}</code>	entoure de guillemets
<code>{{ .Values.port int }}</code>	force le type entier
<code>{{ include "nginx.labels" . indent 4 }}</code>	inclut + indente

```
tag: {{ .Values.image.tag | default "1.27" | quote }}
# → tag: "1.27"
```

7. Les valeurs imbriquées avec `with`

`with` change le contexte courant pour alléger l'écriture :

```
{{- with .Values.resources }}
resources:
  requests:
    cpu: {{ .requests.cpu }}
    memory: {{ .requests.memory }}
{{- end }}
```

8. Déboguer le rendu sans rien installer

La **commande la plus utile** quand on développe un chart :

```
helm template nginx-prod ./nginx # rend tous les manifestes
helm template nginx-prod ./nginx -f values-prod.yaml # avec des valeurs précises
helm install nginx-prod ./nginx --dry-run --debug # simulation complète
```

`helm template` affiche le YAML **final** (valeurs remplacées) **sans toucher au cluster** : on voit immédiatement ce qui serait appliqué.

À retenir : `{{ .Values.x }}` pour injecter, `if / range` pour la logique, `|` pour transformer, et `helm template` pour vérifier. C'est tout l'art du chart : un modèle générique, des valeurs spécifiques.

Releases : installer, mettre à jour, rollback

Une fois le chart prêt, on entre dans le **cycle de vie** : installer une release, la mettre à jour, et — point fort de Helm — **revenir en arrière** en cas de problème.

Le cycle de vie d'une Release (révisions)



Helm garde l'historique : `helm history nginx`. Un upgrade raté ? `helm rollback` en une commande.

Chaque install/upgrade crée une nouvelle révision ; rien n'est jamais perdu.

Chaque install/upgrade crée une révision ; helm rollback revient à n'importe laquelle.

1. Installer une release

```
helm install nginx-prod ./nginx
```

- `nginx-prod` : le **nom de la release** (libre, unique dans le namespace).
- `./nginx` : le chart (dossier local, ou `repo/chart`, ou archive `.tgz`).

```
helm install nginx-prod ./nginx \
--namespace prod --create-namespace \
-f values-prod.yaml
```

Vérifier :

```
helm list -n prod
kubectl get all -n prod      # les objets créés par la release
```

2. Mettre à jour une release

On modifie le chart ou les valeurs, puis :

```
helm upgrade nginx-prod ./nginx --set image.tag=1.28
```

Chaque `upgrade` crée une **nouvelle révision**. Helm calcule la différence et applique le changement (en s'appuyant sur le rolling update des Deployments → **sans coupure**).

```
# Pratique : installer si absent, sinon mettre à jour
helm upgrade --install nginx-prod ./nginx -f values-prod.yaml
```

`upgrade --install` est le réflexe en CI/CD : la commande est **idempotente** — elle installe la première fois, met à jour les fois suivantes.

3. L'historique des révisions

Helm **mémore** chaque révision d'une release.

```
helm history nginx-prod
```

REVISION	STATUS	CHART	APP VERSION	DESCRIPTION
1	superseded	nginx-1.0.0	1.27	Install complete
2	superseded	nginx-1.0.0	1.28	Upgrade complete
3	deployed	nginx-1.0.0	1.28	Upgrade complete

4. Le rollback : revenir en arrière

La nouvelle version est cassée ? On revient à une révision précédente **en une commande**, sans reconstruire ni réécrire quoi que ce soit :

```
helm rollback nginx-prod          # revient à la révision précédente
helm rollback nginx-prod 1        # revient à la révision 1 précise
```

Le rollback crée lui-même une **nouvelle révision** (l'historique n'est jamais réécrit) :

4	deployed	nginx-1.0.0	1.27	Rollback to 1
---	----------	-------------	------	---------------

C'est le filet de sécurité de Helm. Un déploiement raté en production se corrige en quelques secondes. C'est ce qui rend les déploiements fréquents sereins.

5. Inspecter une release

```
helm status nginx-prod          # état + NOTES.txt
helm get values nginx-prod      # valeurs effectivement utilisées
helm get manifest nginx-prod    # YAML réellement appliqué
helm get values nginx-prod --revision 2  # valeurs d'une révision passée
```

6. Désinstaller

```
helm uninstall nginx-prod      # supprime TOUS les objets de la release
helm uninstall nginx-prod --keep-history  # garde l'historique (rollback possible)
```

Un seul `uninstall` retire proprement l'ensemble (Deployment, Service, ConfigMap...) — pas besoin de supprimer les objets un par un.

7. Options utiles de cycle de vie

Option	Effet
<code>--dry-run</code>	simule sans appliquer
<code>--atomic</code>	en cas d'échec d'upgrade, rollback automatique
<code>--wait</code>	attend que les ressources soient prêtes avant de rendre la main
<code>--timeout 5m</code>	délai d'attente max
<code>--namespace</code> / <code>--create-namespace</code>	cibler/créer un namespace

```
# Déploiement « tout ou rien » : si ça échoue, on revient à l'état d'avant
helm upgrade --install nginx-prod ./nginx --atomic --wait --timeout 5m
```

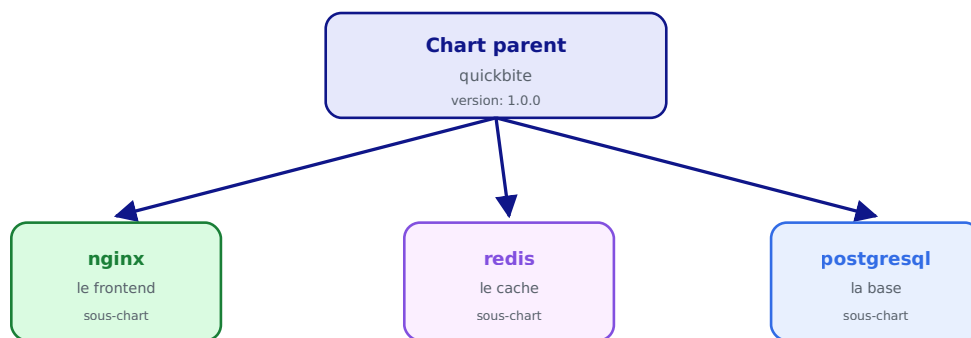
À retenir : `install` → `upgrade` → (`rollback` si besoin) → `uninstall` . Chaque étape est tracée en révisions. Avec `--atomic --wait` , un upgrade raté se répare **tout seul** . C'est la gestion de cycle de vie qui manquait à `kubectl` .

Dépendances & repositories

Une application réelle se compose souvent de **plusieurs briques** : le frontend nginx, un cache Redis, une base de données... Helm permet de les **assembler** via les dépendances, et de **réutiliser** des charts publics via les repositories.

Les dépendances : un chart qui en embarque d'autres

Chart.yaml > dependencies — déclarez, helm dependency update les télécharge



On installe le parent, Helm déploie tout l'ensemble cohérent en une seule Release.

Un chart parent déclare des sous-charts ; on installe le tout en une seule Release.

1. Les repositories : réutiliser des charts existants

Avant d'écrire un chart, vérifiez s'il existe déjà. Les repositories publics regorgent de charts maintenus et éprouvés.

```
# Ajouter des repositories
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update                                # rafraîchir les index

# Lister / chercher
helm repo list
helm search repo nginx
helm search hub wordpress                      # chercher sur l'Artifact Hub (tous repos)
```

Installer un chart public directement :

```
helm install mon-nginx bitnami/nginx --set replicaCount=3
```

Artifact Hub (artifacthub.io) est l'annuaire central des charts publics. On y trouve presque toutes les briques courantes : bases de données, monitoring, ingress, etc.

2. Inspecter un chart distant avant de l'installer

```
helm show chart bitnami/nginx      # métadonnées
helm show values bitnami/nginx     # toutes les valeurs configurables
helm pull bitnami/nginx --untar    # télécharger et décompresser pour l'étudier
```

3. Déclarer des dépendances

Un chart peut **dépendre** d'autres charts (ses **sous-charts**). On les déclare dans `Chart.yaml` :

```
# Chart.yaml du chart parent "quickbite"
apiVersion: v2
name: quickbite
version: 1.0.0
dependencies:
  - name: nginx
    version: "15.x.x"
    repository: https://charts.bitnami.com/bitnami
  - name: redis
    version: "18.x.x"
    repository: https://charts.bitnami.com/bitnami
    condition: redis.enabled      # activable/désactivable
```

Télécharger les dépendances dans `charts/` :

```
helm dependency update quickbite  # remplit charts/ et écrit Chart.lock
helm dependency list quickbite    # état des dépendances
```

4. Configurer les sous-charts depuis le parent

Les valeurs d'un sous-chart se surchargent **sous une clé portant son nom**, dans le `values.yaml` du parent :

```
# values.yaml du parent
nginx:                                # ← valeurs passées au sous-chart "nginx"
  replicaCount: 3
  image:
    tag: "1.27"

redis:                                # ← valeurs passées au sous-chart "redis"
  enabled: true
  auth:
    enabled: false
```

Puis une seule commande déploie **tout l'ensemble** de façon cohérente :

```
helm install quickbite ./quickbite
# → installe nginx + redis (+ le parent) en UNE release
```

5. Le verrouillage des versions : `Chart.lock`

`helm dependency update` génère un `Chart.lock` qui **fige** les versions exactes téléchargées (comme `package-lock.json`). On le **committe** pour des installations **reproductibles**.

```
helm dependency build quickbite      # réinstalle EXACTEMENT les versions du Chart.lock
```

6. Packager et publier son propre chart

```
helm package nginx                  # crée nginx-1.0.0.tgz
helm lint nginx                     # valider avant publication
```

On peut ensuite héberger l'archive et un `index.yaml` sur :

- un serveur HTTP statique (GitHub Pages, S3...) ;
- un **registry OCI** (Docker Hub, GHCR, Harbor...), désormais standard :

```
helm push nginx-1.0.0.tgz oci://ghcr.io/mon-orga/charts
helm install nginx-prod oci://ghcr.io/mon-orga/charts/nginx --version 1.0.0
```

OCI : Helm 3 sait stocker les charts dans les **mêmes registries que les images Docker**. Un seul endroit (GHCR, Harbor...) pour vos images **et** vos charts.

7. Récapitulatif des commandes

```
helm repo add / update / list / search
helm show chart|values <repo>/<chart>
helm dependency update|build|list <chart>
helm pull <repo>/<chart> --untar
helm package <chart>
helm push <chart>.tgz oci://<registry>
```

À retenir : ne réinventez pas la roue. Réutilisez des charts publics (repositories), assemblez vos briques en sous-charts (dépendances), figez les versions (`Chart.lock`) et publiez vos charts comme des artefacts (OCI).

Bonnes pratiques, débogage & checklist

Un bon chart est **lisible, testable et sûr**. Voici les pratiques qui font la différence, les commandes de débogage et une checklist finale.

1. Tester avant d'appliquer

Ne jamais installer un chart sans l'avoir validé. Trois filets de sécurité :

```
helm lint ./nginx # 1) structure et erreurs de syntaxe
helm template nginx-prod ./nginx # 2) voir le YAML rendu (valeurs remplacées)
helm install nginx-prod ./nginx --dry-run --debug # 3) simulation complète
```

Commande	Vérifie
<code>helm lint</code>	la validité du chart (champs requis, conventions)
<code>helm template</code>	le rendu final des manifestes
<code>--dry-run --debug</code>	la simulation contre le cluster (sans rien créer)

2. Bien gérer les valeurs

- **Documenter** chaque valeur dans `values.yaml` avec un commentaire.
- Fournir des **défauts sensés** : le chart doit s'installer sans surcharge.
- Un fichier de valeurs **par environnement** : `values-dev.yaml`, `values-prod.yaml`.
- Préférer `-f values-prod.yaml` à de longues rafales de `--set` (versionnable, relisible).

```
helm upgrade --install nginx-prod ./nginx -f values-prod.yaml
```

3. Versionner correctement (SemVer)

- Incrémenter `version` (du chart) à **chaque modification** du chart.
- Mettre à jour `appVersion` quand l'application embarquée change.
- Suivre le **versionnement sémantique** : `MAJEUR.MINEUR.CORRECTIF`.

Changement	Incrémenter
Correction sans impact	CORRECTIF (1.0.1)
Nouvelle option rétro-compatible	MINEUR (1.1.0)
Changement cassant (valeurs renommées...)	MAJEUR (2.0.0)

4. Sécurité

- **Ne jamais** committer de secrets en clair dans `values.yaml`. Utiliser des **Secrets** Kubernetes, ou des outils comme **helm-secrets** / **Sealed Secrets** / un coffre externe.
- Définir **requests/limits** par défaut dans le chart.
- Épingler les **versions d'images** (`tag: "1.27"`), jamais `latest`.
- Fixer les **versions des dépendances** et committer `Chart.lock`.

5. Lisibilité des templates

- Factoriser le nommage et les labels dans `_helpers.tpl` (`{{ include "nginx.labels" . }}`).
- Utiliser `{{- et -}}` pour **maîtriser les espaces** et les lignes vides.
- Garder une indentation cohérente (`| nindent 4`).
- Rendre les fonctionnalités **optionnelles** avec `if` + une valeur `enabled`.

```
metadata:
  labels:
    {{- include "nginx.labels" . | nindent 4 }}
```

6. Débogage : les pannes courantes

Symptôme	Cause probable	Solution
<code>helm install</code> échoue sur le YAML	erreur de template / indentation	<code>helm template</code> pour voir le rendu
Valeur non prise en compte	mauvais chemin dans <code>.Values</code>	<code>helm get values <release></code>
<code>another operation in progress</code>	release bloquée en <code>pending</code>	<code>helm rollback</code> ou <code>--force</code>
Upgrade casse tout	changement non rétro-compatible	<code>helm rollback</code> , ou installer avec <code>--atomic</code>
Sous-chart absent	<code>charts/</code> non rempli	<code>helm dependency update</code>
Espaces/lignes en trop	gestion des whitespaces	utiliser <code>{{- et -}}</code>


```
helm template ./nginx | kubectl apply --dry-run=client -f - # double validation
helm get manifest <release>                               # ce qui est réellement déployé
```

7. Helm en CI/CD

Le combo gagnant pour un pipeline (à enchaîner après le build d'image) :

```
helm lint ./nginx
helm upgrade --install nginx-prod ./nginx \
  -f values-prod.yaml \
  --set image.tag=$GIT_SHA \
  --atomic --wait --timeout 5m
```

- `--install` rend la commande **idempotente**.
- `--set image.tag=$GIT_SHA` déploie **exactement** l'image qu'on vient de construire.
- `--atomic --wait` : déploiement « tout ou rien », rollback auto si échec.

8. Checklist d'un chart de qualité

- [] `helm lint` passe sans erreur.
- [] `helm template` produit un YAML correct (vérifié visuellement).
- [] Toutes les valeurs sont **documentées** dans `values.yaml` avec des défauts sensés.
- [] Un fichier de valeurs **par environnement**.
- [] **requests/limits** définies ; images **taguées** précisément.
- [] **Aucun secret** en clair dans le chart.
- [] `version` / `appVersion` à jour ; dépendances figées (`Chart.lock`).
- [] Labels et nommage factorisés via `_helpers.tpl`.
- [] Déploiement testé avec `--dry-run`, puis `--atomic --wait` en réel.

Conclusion

Avec un seul chart — celui de **nginx** — vous avez parcouru tout Helm : les **charts**, les **releases** versionnées, le **templating** (`{{ .Values }}`, `if`, `range`, fonctions), le cycle de vie (**install** / **upgrade** / **rollback**), les **dépendances** et les **repositories**, jusqu'aux bonnes pratiques de production.

Helm transforme des piles de YAML en **applications packagées, paramétrables et versionnées**. Combiné à un pipeline CI/CD et à GitOps, c'est la base d'un déploiement Kubernetes professionnel.

Pour aller plus loin : les **library charts** (mutualiser du template entre charts), **helmfile** (orchestrer plusieurs releases), les **tests de chart** (`helm test`), et l'intégration de Helm dans **Argo CD** pour le GitOps.